

Development and Design of a Microblogging Platform: Vibe

Tellez Melo David Santiago
Lopez Hernandez Johan Nicolas
Universidad Distrital Francisco José de Caldas
dstellezm@udistrital.edu.co
jnlopezh@udistrital.edu.co

Abstract—The widespread use of social networks in modern society presents both an opportunity and a challenge for creating platforms that are efficient, user-friendly, and secure. This paper presents the development of *Vibe*, a lightweight microblogging application designed with a modular, object-oriented architecture that promotes scalability and maintainability. The final system successfully implements core features such as user registration, content posting with media support, personalized and global feeds, profile management, and social interactions including likes and follows, offering a comprehensive yet simplified social experience.

Index Terms—Microblogging, Social Networks, Object Oriented Programming, Java.

I. INTRODUCTION

In today's digital landscape, microblogging platforms play a crucial role in facilitating real-time communication, community building, and information dissemination. These platforms—such as Twitter and Threads—operate as complex, dynamic systems that integrate various components for user input, data persistence, interaction, and visualization. From a systems perspective, they exemplify the convergence of distributed architecture, user-centered design, and continuous information flow.

The goal of this research is not merely to replicate the interface of a social media platform, but to model and construct a simplified version of its architecture. Specifically, we aim to design a desktop-based microblogging system—named *Vibe*—that demonstrates the core operational mechanisms of such platforms. Our primary focus is on system design and modular implementation, emphasizing object-oriented programming principles and architectural clarity.

To guide this development, we adopted the Model-View-Controller (MVC) architectural pattern, enabling a clear separation between user interface, application logic, and data management. Each module was designed to maintain low coupling and high cohesion, following fundamental principles of software engineering. This modularity is essential not only for maintainability and testing, but also for ensuring a realistic simulation of a multi-user digital communication platform.

Several conceptual tools were employed to structure the system design, including UML class diagrams and CRC (Class-Responsibility-Collaboration) cards. These tools provided clarity in role assignment and helped formalize the responsibilities of each component within the system [1].

Furthermore, the layered design approach allowed for iterative testing and integration of each functional unit, reflecting best practices in modern software architecture [2].

Ultimately, this project offers a pedagogical and replicable framework for understanding how microblogging systems operate as structured information systems. By simulating key aspects such as user authentication, content publication, feed personalization, and reaction management, *Vibe* serves as a compact but illustrative model for exploring distributed social platforms from both a technical and systemic standpoint.

II. METHODS AND MATERIALS

The development of *Vibe* was guided by object-oriented principles to structure a microblogging system that supports user interaction, content publication, and personalized feeds. The architecture was designed using class diagrams, CRC cards, and usage flow illustrations, providing a clear blueprint for implementation and collaboration.

System architecture is divided into three main packages: `model`, `controller`, and `view`, following an MVC-like organization. The `model` package defines the data entities and their behaviors. The central class, `User`, represents the platform's users and manages their information, including name, age, email, password, profile image, published content, followers, and following. Sensitive attributes are encapsulated and exposed through controlled public methods. Each user can create posts, reply to other users, like content, and view their own timeline.

All published content is modeled through the `Tuit` interface, implemented by two classes: `Post` and `Reply`. The `Post` class stores the content of a tweet, its author, publication date, image (if any), external link, and the number of likes. It is also responsible for associating content with the original author. The `Reply` class represents responses to other tweets and includes a reference to the original `Tuit`, allowing nesting of replies.

To manage the user's visible content stream, the `Feed` class was designed. It builds a list of relevant `Tuit` objects based on whether the feed is global or user-specific. For user-specific feeds, it includes only posts from his account, while the global feed aggregates all posts. The `Feed` class handles sorting and filtering, keeping the content dynamic and relevant.

The controller package orchestrates the application logic. The `AppController` class is the central hub connecting the user interface and model. It handles actions like posting, replying, liking, following/unfollowing users, and viewing profiles. The `FileManager` class is responsible for persistent storage, handling JSON serialization of users and tweets. It also manages image paths and user profile pictures. Meanwhile, the `UserManager` class maintains the collection of users and provides login and registration functionality.

The view package implements the graphical user interface (GUI) using Java Swing. The main class, `VibeGUI`, defines the layout for navigation, post creation, and feed visualization. It supports two types of feed: global and personal, and includes functionalities to upload images, share links, react to tweets, and view user profiles. GUI components were carefully designed to match the structure of the system and ensure a clear user experience. Auxiliary interfaces like `LoginGUI`, `RegisterGUI`, and `StartGUI` manage the authentication flow and transitions between screens.

To support system planning, we designed several CRC cards that define class responsibilities and their collaborators (see Figure 1). For example, the `User` class is responsible for managing user data and collaborating with `AppController` and `Feed`. The `Tuit` interface unifies content types and ensures consistent behavior across posts and replies.

We also constructed a UML class diagram to capture class hierarchies, associations, and responsibilities (see Figure 2). This diagram served as a structural reference during implementation. To ensure user-centered design, we illustrated key usage scenarios in diagrams, including posting tweets, following users, and interacting with the feed (see Figure 3). These diagrams helped validate feature completeness and navigation flow.

Class	FileManager	Class	VibeGUI	Interface	Tuit
Responsibilities	- Read and write persistent data to disk (JSON, images). - Representing of users, tweets, and replies.	Collaborations	- UserManager - AppController - Feed - Reply	Responsibilities	- Display the user interface (feed, login, register, etc). - Handle user registration and login. - Handle user profile updates and follow/unfollow actions. - Handle user data storage and retrieval.
Class	Reply	Class	AppController	Class	Post
Responsibilities	- Represent a reply to another tweet. - Store content, timestamp, likes, and replies. - Handle references to original tweet. - React to author.	Collaborations	- Tuit (parent) - User - AppController	Responsibilities	- Represent a standalone tweet (post). - Store content, timestamp, likes, and replies. - React to author.
Class	User	Class	AppController	Class	LoginGUI
Responsibilities	- Store user attributes (name, email, password, profile picture, etc). - Handle user registration and login. - Handle user profile updates and follow/unfollow actions. - Handle user data storage and retrieval.	Collaborations	- FileManager - AppController	Responsibilities	- Allow users to log in with email and password. - Validate credentials and redirect to main app.
Class	Feed	Class	RegisterGUI	Class	StartGUI
Responsibilities	- Build and return the general feed (all posts). - Sort post chronologically.	Collaborations	- User - AppController - FileManager	Responsibilities	- Let new user sign up with basic data and profile. - Create and save the new user.
				Responsibilities	Display the welcome screen with "Login" and "Register" options.

Figure 1. CRC cards defining class responsibilities and collaborators.

This architecture allows easy extension in future phases, such as adding search filters, improving data persistence with databases, or implementing notifications. All components were designed with cohesion and modularity in mind, ensuring maintainability and clarity throughout development.

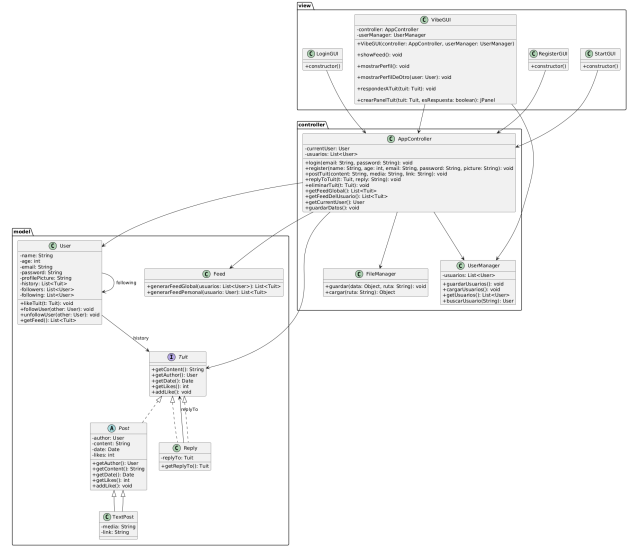


Figure 2. UML diagram showing main classes and their relationships.

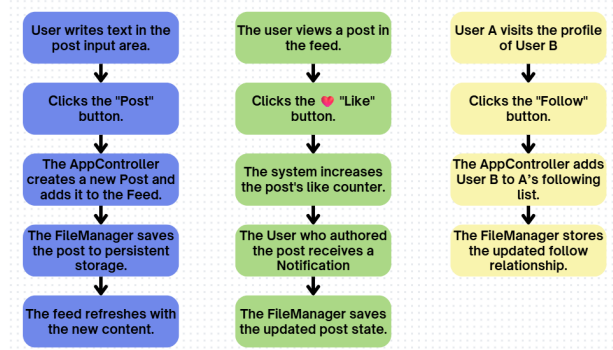


Figure 3. Usage flow diagrams: posting, liking, following, and feed navigation.

III. RESULTS

To evaluate the functionality and consistency of *Vibe*, we conducted a complete series of unit and integration tests throughout the development process. These tests were essential to verify that all main components interacted correctly and that the application behaved as expected under typical usage scenarios.

Unit tests were performed on key methods of the `User`, `Post`, `Reply`, and `Feed` classes. The goal was to ensure that actions such as user registration, authentication, following/unfollowing, publishing posts, liking content, and replying to tweets executed correctly and updated the internal state of the system. Each method was tested with both valid and invalid inputs, confirming that the system handles edge cases gracefully and prevents invalid data manipulation.

Integration tests focused on validating the cooperation between the model, controller, and view layers, specifically the coordination performed by the `AppController`. These tests included creating a post and immediately reflecting it in the feed, replying to posts and showing replies with proper

indentation, switching between global and personal feeds, and verifying correct profile rendering. Special attention was given to transitions between screens (e.g., from login to feed, or from a tweet to a user profile), ensuring that user context was preserved and data remained consistent.

A crucial part of testing was the evaluation of data persistence. After creating users and publishing several tweets with likes and replies, the application was closed and reopened to verify that all data was preserved. The `FileManager` class successfully serialized and deserialized the complete state of the system using JSON, including profile pictures, tweet history, and follow relationships.

From a user experience perspective, we verified the functionality of all user interfaces. The application supports complete flows for account creation, login, tweet publishing (including image and link upload), liking and replying, viewing user profiles, and following or unfollowing other users. The GUI design was tested on multiple screen sizes, confirming that all interactive elements (buttons, text inputs, tweet panels) behaved responsively and as intended. No critical usability issues were found during testing.

Overall, the platform proved to be stable and functionally complete for its current scope. The results confirm that the implemented architecture supports consistent user interaction, smooth navigation, and reliable content management, laying the foundation for future feature extensions and performance improvements.

IV. CONCLUSIONS

The development of *Vibe* demonstrates the viability and effectiveness of a modular, object-oriented approach to building a microblogging platform. By dividing responsibilities clearly across model, view, and controller layers, and relying on persistent storage mechanisms, the application achieves both functional completeness and maintainable architecture.

The final system supports the essential features expected in a microblogging service: user registration, login, tweet posting with images and links, likes, replies, following relationships, and user profiles. All features were integrated into a graphical user interface that reflects a coherent and intuitive user experience.

From a software engineering perspective, the use of UML diagrams, CRC cards, and usage flow representations contributed significantly to the clarity and consistency of the project's architecture. Testing confirmed the system's robustness, with all critical user interactions working correctly and data persisting reliably across sessions.

In summary, the project not only fulfilled its initial objectives but also set a scalable foundation for future improvements. Features such as real-time interaction, mobile adaptation, or third-party API integration could be explored in subsequent development phases. The design and testing process followed throughout the project provide a replicable framework for similar system-level applications.

REFERENCES

- [1] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall, 2004.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.